Program #4

Purpose

Demonstrate the ability to create and use subclasses and inheritance. This includes overriding method behaviors and using polymorphism.

Assignment

Create an Aircraft class that has several properties that are common to all aircraft (Ex: number of engines, seat capacity). You define the name of the Class and the actual fields. The fields MUST BE PRIVATE!!! You must define a constructor that allows you to provide at least some of the field values used for an aircraft. You must define a `printCharacteristics()` method.

Create three subclasses for specialized types of aircraft. An example might be a Fighter, Acrobat, and Freight. You do not have to use these three choices. You define the names of the classes. Each specialized craft should have additional properties that are common to just that type of specialized craft. These fields MUST BE PRIVATE. For each derived class, you must define a constructor that allows you to provide at least some of the field values used for these subclass aircraft. You must define any getter/setter methods you need.

Make a SINGLE C++ ***vector*** of Aircraft pointers. You will populate this list by reading commands from an input file. This vector will hold pointers to *any type* of Aircraft including your three derived classes.

The commands in the file will cause you to create your Aircraft objects or print them using the `printCharacteristics()` method. **This method must be overridden in each subclass**. The output should print a well formatted description of all the specific and inherited information for each craft. (Make sure you up on virtual functions and virtual destructors).

Requirements

Your code must exhibit the use of Polymorphism.

Do not create separate arrays for objects of each class.

Your code must exhibit the use of subclasses
Your code must exhibit the use of overridden methods
Your code must exhibit the use of a parameterized STL vector

Deliverables

You must submit your homework through ELearning. You must include UML diagrams of your classes, a Class Hierarchy diagram, your .h and .cpp files, and a commented version of your input file. Each Class MUST be implemented in a separate .cpp file that has a corresponding .h file.

Notes

No late homework is accepted.

A nice UML tool is available here: https://www.gliffy.com

Input File Format

The input file is a text file that contains an unknown number of **`commands`**. **`Commands`** are strings that determine what operations should be performed. If the commands have arguments, they are listed after the command. Multiple arguments will be comma separated. Spaces, tabs, and anything following // can be ignored.

This assignment requires you to create 4 separate classes: your aircraft class, your subclass 1, your subclass2, and your subclass 3. In the commands section of the file, these classes will be referred to by number:

Class	Referred to using
Your Aircraft Class	0
Your subclass 1	1
Your subclass 2	2
Your subclass 3	3

The commands all have to do with creating aircraft and printing information about those aircraft. Since you are defining your own classes and subclasses, you will also have to define the arguments that go with the commands. Your commands must provide enough information so that you can execute them. The commands are:

String Command	Description
n	Create a new aircraft. The first arg is the aircraft type. **You must define the rest of the arguments needed to create your aircraft of that type**: int aircraft type (from the "Referred to using") ... (you define the rest for each aircraft type) Each aircraft should be placed in the aircraft vector. The first aircraft you create will be numbered 1, the second 2, etc...
p	Print the characteristics of an aircraft in your list. This command takes one argument which represents the actual Aircraft to print. If the argument has the value 0, you print information for **all** the aircraft. Otherwise, go through the entire list of aircraft and only print the information for the single type of aircraft referred. Remember, the first Aircraft created is numbered 1.

Your code must check for proper information for each command. Errors should be handled by printing a message and ignoring the line. Examples of errors include, but are not limited to: unknown commands, commands without the proper number of arguments, commands with arguments that do not make sense, and commands that have the wrong types of arguments.

Example Input File

```
// Commands information:
//   n - create a new aircraft. If the first arg is 0, then parameters are
//                int: number of engines
//                int: seat capacity
//
// if the first arg is 1, then the parameters are:
//                int: number of engines
//                int: seat capacity
//                String: range and speed description
//
// if the first arg is 2 then the parameters are:
//                String: Manufacturer
//                String: Performance capability description
//
// if the first arg is 3 then the parameters are:
//                int: number of engines
//                String: Freight Company
//                String: Cargo Capacity

n 0, 2, 6          // New Aircraft: 2 engines, seats 6
```

```
n 0, 4, 200       // New Aircraft: 4 engines, seats 200
n 1, 1, 1, "Top Speed: Mach 1.2,  Max Range: 400 Miles"
n 1, 2, 2, "Top Speed: Mach 2,  Max Range: 1200 Miles"
n 2, "Zivko Edge 540", "Cruise: 333 km/h, Max Roll Rate: 420 degrees/sec, G-limit: 10"
n 2, "Pitts Special S-1S", "180HP,  certified S-1C for competition aerobatics"
n 3, 4, "UPS", "100 UPS Shipping containers"
n 3, 4, "FedEx", "14,000 pounds"
p 0
p 1
p 2
p 3
```